Engineering White Paper | Performance, Coverage & Sustainability Validation

Hybrid Load, Endpoint Coverage, and SQLite Concurrency Testing Methodology for SD3 Alumni

A Two-Phase Parallel Orchestration Architecture Combining Fail-Fast JMeter Contract Verification, Think-Time-Modulated Stepped Load, Behavioral Locust Personas, and Automated Multi-Modal Reporting

Target Environment:	**Database Engine:**	**Uplink Envelope:**	**Effective Date:**
`https://najealp.top`	SQLite (`DELETE` mode)	70 Mbps ($\approx$ 8.75 MB/s)	November 11, 2026

Abstract

Conventional API stress tests frequently mischaracterize production readiness because of three methodological failures: silent HTTP `401 Unauthorized` cascades that inflate synthetic throughput without exercising database paths, unpaced burst loops whose samples collapse to start/end boundaries of time-series charts, and homogeneous read-only requests that overlook write-lock contention. This white paper formalizes the hybrid testing methodology implemented for the SD3 Alumni platform (`test/load/`). The architecture separates validation into a deterministic, fail-fast **Phase 1** credential pre-flight and 191-sample endpoint/CRUD coverage gate in Apache JMeter (`benchmark.jmx`), followed by a **Phase 2** parallel execution combining think-time-paced stepped JMeter load tiers with stateful, multi-role Locust personas (`locustfile.py`), and concluding with a **Phase 3** automated telemetry compiler (`generate_report.py` and `export_reports_pdf.mjs`). Empirical verification against `https://najealp.top` demonstrates 0% HTTP failure across 191 coverage requests, 499 stepped JMeter load samples (APDEX = 0.953, $p_{50} = 244$ ms, $p_{95} = 886$ ms), and 216 parallel Locust behavioral transactions ($p_{50} = 280$ ms, $p_{95} = 1000$ ms), while isolating synchronous `bcrypt` hashing and SQLite `DELETE`-mode write locking as the principal drivers of tail latency.

Contents

1 System Under Test and Operational Envelope **2**
- 1.1 Application Stack and Persistence Layer . . . 2
- 1.2 Deliberate SQLite Journal Policy and Physical Bandwidth Ceiling . . . 2

2 Methodological Foundations and Threat Model **3**

3 Two-Phase Parallel Orchestration Architecture **4**

4 Phase 1: Credential Verification and Full Endpoint Coverage Matrix **4**
- 4.1 Pre-Flight Credential Signing and Token Validation . . . 4
- 4.2 Comprehensive 186-Sampler Functional & CRUD Coverage Matrix . . . 5

5 Phase 2: Parallel Stepped Concurrency and Realistic Personas **5**
5.1 Engine A: Apache JMeter Stepped Concurrency (`-SkipCoverage`) 5
5.2 Engine B: Locust Headless Behavioral Personas 6
5.2.1 Persona Pool Pre-Flight and Auto-Provisioning 6
5.3 Analytical Timing and APDEX Models 7

6 Phase 3: Multi-Artifact Telemetry and Report Synthesis **7**
6.1 Five-Sheet Executive Workbook (`generate_report.py`) 7
6.2 Automated Multi-Tab PDF Compiler (`export_reports_pdf.mjs`) 7

7 Empirical Verification Results **8**
7.1 Cross-Engine Dry-Run Telemetry (`https://najealp.top`) 8
7.2 Route-Level Bottleneck Attribution 9

8 Operational Governance and Reproducibility **9**
8.1 Database Hygiene and Rate-Limiter Controls 9
8.2 Standardized Execution Profiles 10

1 System Under Test and Operational Envelope

1.1 Application Stack and Persistence Layer

The SD3 Alumni platform exposes a monolithic Node.js backend (`api/index.js`) serving both JSON REST routes and a compiled React 19 Single-Page Application (SPA) catch-all handler:

- **HTTP & Routing Layer:** Express 5 (`express ^5.3.0`) with JWT authentication (`jsonwebtoken ^9.0.3`) supporting dual extraction from `Authorization: Bearer` headers and HTTP cookies (`token` / `adminToken`). Role separation is enforced by middleware: `authenticateToken` rejects missing tokens with HTTP `401` and admin tokens with HTTP `403`, whereas `authenticateAdminToken` enforces `role === "admin"`.
- **ORM & In-Process Database:** Prisma ORM (`@prisma/client ^7.10.1`) bound through `@prisma/adapter-better-sqlite3` to native `better-sqlite3` (`^12.11.1`). Generic REST routes (`GET/POST/PATCH/PUT/DELETE /api/:table`) share dynamic filtering, sorting, and relational hydration via `queryParser.js` (`_where`, `_sort`, `_order`, `_include`, `_page`, `_per_page`).
- **Cryptographic Workloads:** User authentication relies on `bcryptjs` (`^3.0.3`, cost factor 10). Crucially, inspection of `api/index.js` reveals an asymmetry: login paths invoke asynchronous `await bcrypt.compare(...)`, whereas write endpoints—`POST /api/auth/register`, `POST /api/auth/reset-password`, and `POST /api/admin/users`—execute synchronous `bcrypt.hashSync(password, 10)`, blocking the Node.js event loop for the key-derivation duration.
- **Media, External Integrations & Rate Limiters:** Image uploads pass through `multer` and `sharp` (`POST /api/upload`); donation checkout flows call PayMongo (`POST /api/donations/checkout` and `/api/donations/webhook`). During test campaigns, `authLimiter` and `contentLimiter` (`express-rate-limit`) set `skip: () => true` to measure server capacity rather than IP-throttling thresholds.

1.2 Deliberate SQLite Journal Policy and Physical Bandwidth Ceiling

Per project architectural constraints (`api/index.js`), the database explicitly maintains rollback journal mode rather than Write-Ahead Logging:

Listing 1: Active SQLite concurrency pragmas initialized at backend startup (`api/index.js`).

```
PRAGMA journal_mode = DELETE;
PRAGMA busy_timeout = 10000;
```

In `DELETE` mode, SQLite permits concurrent shared reads when no writer commits, but acquiring an exclusive commit lock blocks all concurrent readers and writers until the rollback journal is unlinked, bounded by the 10 s adapter and SQLite `busy_timeout`. Preserving this constraint intentionally lets the test suite observe lock-queue degradation under realistic write pressure instead of masking it.

Simultaneously, `test/load/config.json` constrains the target VPS host with a physical network ceiling of $C_{\text{uplink}} = 70$ Mbits/s (≈ 8.75 MB/s). For relational catalog responses with average wire payload $\bar{S}$ bytes, the theoretical network-bound request ceiling $\lambda_{\max}$ follows:

$$\lambda_{\max} = \frac{C_{\text{uplink}} \times 10^6}{8 \cdot \bar{S}} \quad [\text{req/s}]. \tag{1}$$

For paginated relational feeds averaging $\bar{S} \in [15, 25]$ KB, link saturation occurs near 350–580 req/s, meaning load tests must separately resolve WAN queueing from SQLite lock contention.

2 Methodological Foundations and Threat Model

During suite evolution, three structural pathologies of naive load tests were identified and systematically eliminated:

1. **The Silent `401` Throughput Mirage:** If seeded accounts or JWT tokens fail before load generation, protected routes short-circuit in middleware before touching Prisma or SQLite. Throughput rises artificially while real application logic remains untested. Our methodology makes token verification and automatic fallback provisioning mandatory prerequisites, treating *any* `HTTP 401` across Phase 1 or Phase 2 as a hard suite failure (exit code `2`).

2. **Edge-Hugging Time-Series Collapse ("Request Slamming"):** Closed-loop threads lacking inter-request think times dispatch requests simultaneously at thread spawn and finish together before sampling intervals settle. With JMeter's default 60 s HTML report granularity and zero timer delay, plots such as *Transactions per Second* exhibit points hugging only the left and right edges. Introducing a randomized per-iteration `UniformRandomTimer` (100–500 ms), configuring 1000 ms reporting granularity, and separating setup/coverage samples from load samples restores smooth, physically meaningful per-second waveforms.

3. **Cross-VU State Collisions:** When multiple virtual users target identical primary keys for RSVPs, social connections, or moderation toggles, unique constraints and state-transition races distort error rates with test-harness artifacts rather than genuine server faults. We isolate state via VU-owned disposable resources, dedicated moderation target pools, and deterministic pre/post-test cleanup.

Unpaced Ceiling Probing vs. Paced Realistic Concurrency

The repository preserves two complementary load mechanisms: the legacy Node.js worker engine (`test/load/node_benchmark.js`, paired with `run_suite.ps1`) and the unified JMeter + Locust parallel orchestrator (`run_parallel_suite.ps1`). The Node engine opens up to 2000 HTTP/HTTPS `keepAlive` sockets with zero think time across 20 read endpoints to locate raw network/socket limits quickly, whereas the unified suite pairs paced stepped concurrency (JMeter) with heterogeneous read/write user personas (Locust) to measure end-to-end sustainability.

3 Two-Phase Parallel Orchestration Architecture

The canonical entry point `test/load/run_parallel_suite.ps1` enforces strict environment governance before coordinating three sequential stages (with two parallel load engines inside Phase 2), as illustrated in Figure 1.

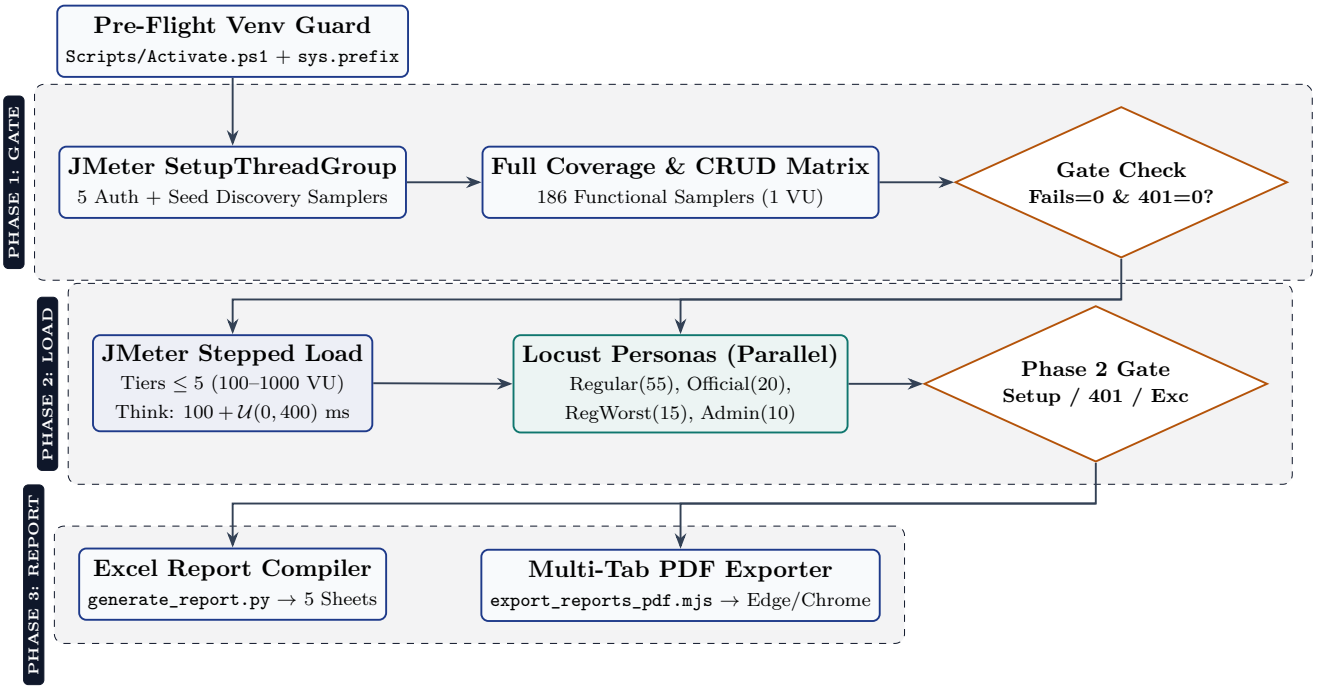


Figure 1: End-to-end orchestration pipeline in `test/load/run_parallel_suite.ps1`. Phase 1 gates Phase 2 load injection; Phase 2 executes JMeter and Locust concurrently; Phase 3 builds consolidated Excel and expanded PDF artifacts.

Before touching any workload engine, `run_parallel_suite.ps1` activates the repository-local Python virtual environment (`Scripts/Activate.ps1`), asserts `$env:VIRTUAL_ENV` matches the repository root, verifies `python` resolves to `Scripts/python.exe`, and confirms `sys.prefix != sys.base_prefix` per `.agents/python.md`. Stale JTL, JSON, log, and HTML directories in `test/load/results/` are purged prior to Phase 1.

4 Phase 1: Credential Verification and Full Endpoint Coverage Matrix

4.1 Pre-Flight Credential Signing and Token Validation

Because `benchmark.jmx` sets `TestPlan.serialize_threadgroups=true`, its `SetupThreadGroup` executes strictly before any subsequent thread group. It contains five samplers:

1. A Groovy `JSR223Sampler` (`[Setup] Sign Credentials, Verify All Tokens & Discover Seed IDs`) that reads candidate accounts from `config.json` (default password `123456`), authenticates Admin, Official, and Regular accounts, provisions missing functional testers via `/api/admin/users` or `/api/auth/reset-password` when absent, and extracts live entity IDs (`sampleBulletinId`, `sampleEventId`, status lookup IDs, categories, degrees) into global JMeter properties.

2. Four explicit HTTP verification samplers validate the minted tokens end-to-end before Phase 1

proceeds: `GET /api/auth/admin/me`, `GET /api/admin/dashboard-stats`, `GET /api/auth/me (Official)`, and `GET /api/auth/me (Regular)`.

4.2 Comprehensive 186-Sampler Functional & CRUD Coverage Matrix

Immediately following setup verification, the `Full Endpoint Coverage & CRUD Matrix` thread group runs as a single serialized thread (`-CoverageOnly` sets stepped tier threads to zero) exercising **186 functional requests** (191 total including the 5 setup samplers):

- **Lookup & Relational Catalog Reads:** All read-only status dictionaries (`userStatuses`, `contentStatuses`, `eventStatuses`, `donationStatuses`, `profileStatuses`, `connectionStatuses`, `degrees`, categories, achievements), `HEAD` requests, paginated feeds, `_include` relational joins, and `_sort`/`_order` ordering checks.
- **Query Parser Operator Coverage:** Targeted probes on `/api/achievements` and `/api/users` verifying `_where` predicates including `contains`, `endsWith`, `startsWith`, `gt`, combined `in+gte+lte`, `notIn`, and nested foreign-key filtering.
- **Isolated CRUD Lifecycles (`POST` → `GET` → `PATCH` → `PUT` → `DELETE`):** Round-trip creation, retrieval, partial/full updates, and deterministic deletion across domain tables (`locations`, `bulletins`, `bulletinLikes`, `comments`, `commentLikes`, `events`, `userRsvps`, `donations`, `clefLogs`, `records`, `notifications`, `userAchievements`, plus admin user/bulletin/event creation).
- **Stateful Workflows, Media & External Gateways:** Multipart fixture upload (`fixture.png` → `POST /api/upload` → `GET /uploads/...`), SPA root and hero asset checks, disposable password-reset verification, all social connection transitions on `/api/connections/action` (`add`, `accept`, `reject`, `remove`, `block`, `unblock`), event conclusion (`POST /api/events/:id/conclude`), and conditional PayMongo checkout/webhook tests toggled by `INCLUDE_EXTERNAL_APIS`.

If `total401 > 0` or `totalFailed > 0`, `run_parallel_suite.ps1` logs offending route breakdowns and aborts immediately without spawning Phase 2 load.

5 Phase 2: Parallel Stepped Concurrency and Realistic Personas

When Phase 1 passes cleanly, Phase 2 launches two complementary engines concurrently so background write/moderation contention overlaps with stepped concurrency scaling.

5.1 Engine A: Apache JMeter Stepped Concurrency (`-SkipCoverage`)

During Phase 2, `run_jmeter.ps1` invokes `benchmark.jmx` with `-JrunCoverage=false`, preserving the 5-sampler auth `SetupThreadGroup` while disabling the 186-sampler coverage thread group. Five sequential stepped thread groups (`TIER1`–`TIER5`) invoke a shared `ModuleController` referencing `Load Mix Test Fragment`:

- **Tier Profiles:** `DryRun` evaluates $V \in \{10, 25\}$ VUs for $D_{\text{tier}} = 10$ s each ($R_{\text{ramp}} = 2$ s); `Full` defaults to $V \in \{100, 250, 500, 750, 1000\}$ VUs for $D_{\text{tier}} = 30$ s ($R_{\text{ramp}} = 10$ s). Thread groups assigned 0 threads are skipped.
- **Weighted Mix & Think-Time Pacing:** A `RandomController` dispatches uniformly across **51 enabled endpoints** (lookups, paginated catalogs, relational `_include` feeds, sorted feeds, admin dashboards, token checks, server time, static assets, and `check-email`/`HEAD`), plus optional write transactions gated by `-IncludeWritesInLoad`. Every iteration pauses for a uniform random think time:

$$T_{\text{think}} \sim \mathcal{U}(t_{\min},\, t_{\min} + \Delta t) = \mathcal{U}(100,\, 500)\ \text{ms}, \qquad \mathbb{E}[T_{\text{think}}] = 300\ \text{ms}. \tag{2}$$

- **Clean Load-Only Dashboard Filtering:** Prior to invoking JMeter's HTML ReportGenerator at `overall_granularity = 1000 ms`, `run_jmeter.ps1` filters the raw JTL stream with a *BOM-free UTF-8* writer (`[System.Text.UTF8Encoding]::new($false)`) retaining only rows matching `^Stepped Tier \d+` into `jmeter_results_load.jtl`. Removing the UTF-8 Byte Order Mark prevents JMeter CSV parser header corruption, while excluding Setup rows keeps initial auth warmup outliers out of load APDEX and percentiles.

5.2 Engine B: Locust Headless Behavioral Personas

Running simultaneously alongside JMeter, Locust executes for $T_{\text{locust}} = \max(15,\ N_{\text{active_tiers}} \cdot D_{\text{tier}} + 4)$ s (24 s in DryRun at 20 VUs spawned at 5 VU/s; 100 VUs at 10 VU/s in Full defaults). To avoid Windows headless `Start-Process` stdin errors in `locust/input_events.py` (`isatty()` on a null handle), `run_parallel_suite.ps1` binds standard input to an empty temporary file cleaned up on exit.

5.2.1 Persona Pool Pre-Flight and Auto-Provisioning

In `on_test_start`, Locust tests Admin, Official (prioritizing `loadtest_jmx_official@najealp.top`), and Regular candidate pools in parallel via `ThreadPoolExecutor(max_workers=10)`. Missing accounts trigger automatic admin-API provisioning (`_provision_and_verify_user`). If any pool remains empty, Locust sets `SETUP_ERROR`, triggers `environment.runner.quit()`, and aborts instead of running failing users. Pre-flight then discovers category IDs and seeds 5 disposable Pending bulletins plus 4 disposable events (2 moderation targets, 2 conclude targets).

Table 1: Behavioral workload distribution and pacing parameters in `test/load/locustfile.py`.

Persona Class	Weight	Wait Time (s)	Primary Stateful Workflow & Isolation Strategy
`RegularUser`	55%	$\mathcal{U}(0.5, 2.0)$	Browses sorted/filtered bulletins & events, relational directories, lookups; creates VU-exclusive event (`own_event_id`) + toggles private RSVP (`own_rsvp_id`); patches own profile bio; posts tracked community bulletins.
`OfficialUser`	20%	$\mathcal{U}(0.3, 1.5)$	Posts approved official bulletins ($w = 4$); manages exclusive official event RSVP lifecycle ($w = 3$); inspects relational bulletin catalogs ($w = 2$).
`RegistrationWorstCase`	15%	$\mathcal{U}(0.8, 2.5)$	Sends `POST /api/auth/register` bursts combining synchronous `bcrypt.hashSync` CPU work with multi-table SQLite writes.
`CrazyAdmin`	10%	$\mathcal{U}(0.1, 0.5)$	Aggressively polls `/api/admin/dashboard-stats` and donation summaries/tables while flipping moderation statuses across disposable seeded bulletin/event targets.

Each Locust user validates its session token in `on_start` (`/api/auth/me` or `/api/auth/admin/me`), cleans up its VU-owned RSVP and event in `on_stop`, and relies on `on_test_stop` to purge all `CREATED_BULLETIN_IDS` and `CREATED_EVENT_IDS`, serialize per-route metrics plus runner exception counts (`exceptionsCount`) into `locust_metrics.json`, and trigger strict failure gates if `setupError`, `count401 > 0`, `totalErrors > 0`, or `exceptionsCount > 0`.

5.3 Analytical Timing and APDEX Models

For closed-loop paced virtual users of count V experiencing mean server response latency $\bar{L}_{\text{resp}}$ and expected think time $\mathbb{E}[T_{\text{think}}]$, Little's Law predicts steady-state tier throughput:

$$\lambda(V) = \frac{V}{\bar{L}_{\text{resp}} + \mathbb{E}[T_{\text{think}}]}. \tag{3}$$

User-perceived responsiveness is summarized via the standard Application Performance Index (APDEX) with JMeter thresholds $T = 500$ ms (satisfied) and $F = 1500$ ms (frustrated):

$$\text{APDEX} = \frac{N_{t \le T} + 0.5 \cdot N_{T < t \le F}}{N_{\text{total}}}. \tag{4}$$

6 Phase 3: Multi-Artifact Telemetry and Report Synthesis

6.1 Five-Sheet Executive Workbook (`generate_report.py`)

Configured via `--source` (`auto`, `jmeter`, or `node`), `generate_report.py` compiles `docs/stress_test_report.xlsx` using `openpyxl` (with timestamped fallback filenames if locked by Excel):

1. **Executive Summary & Verdict:** KPI cards (requests, peak RPS, Mbps, error rate, $p_{50}/p_{95}/p_{99}$, % of 70 Mbps link) plus narrative analysis contrasting SQLite's 0 ms in-process read latency against `DELETE`-mode write serialization.
2. **Latency Evolution & Scaling:** Per-tier concurrency metrics paired with a native Excel line chart tracking p_{50}, p_{95}, and p_{99} across stepped VU tiers.
3. **Transaction Rankings:** Routes ordered by cumulative time consumption (`totalTimeMs`) and tagged into *Standard Read*, *Relational Hydration*, *Filtered & Sorted*, and *Admin Aggregation*.
4. **Transaction Variances:** Top-5 HTTP status code distributions per route with per-status min/median/max latencies and error snippets.
5. **Baseline vs. Personas:** Side-by-side matrix contrasting JMeter/Node stepped load with granular Locust persona route timings.

6.2 Automated Multi-Tab PDF Compiler (`export_reports_pdf.mjs`)

To assemble visual Flot time-series graphs from JMeter alongside Locust's ECharts/MUI dashboard into a single archival artifact (`test/load/results/load_test_report.pdf`), `export_reports_pdf.mjs` launches an ephemeral static HTTP server over `test/load/results/` and drives headless Microsoft Edge or Google Chrome via `puppeteer-core`:

- **Iframe Isolation & Anchor-Navigation Bypass:** Each report tab (`index.html`, `OverTime.html`, `Throughput.html`, `ResponseTimes.html`, `CustomsGraphs.html`, `locust_report.html`) loads in an isolated 1440 px-wide `<iframe>` (`deviceScaleFactor: 2`). JMeter's collapsed graph panels use Bootstrap classes `.collapse.out.portlet-content` and lazy Flot initializers; invoking JMeter's built-in `collapse()` helper inside an iframe sets `document.location.href = "#..."` and breaks relative routing. The exporter instead reveals `.collapse.portlet-content` containers via DOM/CSS and directly calls all 14 JMeter `refresh*` functions (`refreshTransactionsPerSecond(true)`, `refreshActiveThreadsOverTime(true)`, etc.).

- **Exact 1-Page-Per-Tab CSS Sizing:** Empty tabs such as `CustomsGraphs.html` (zero graph panels and zero non-zero canvases) are pruned automatically, while remaining tabs receive dynamically computed CSS named `@page` dimensions (`size: 1440px <measuredH>px`) so every dashboard tab exports intact without awkward mid-chart page breaks.

7 Empirical Verification Results

7.1 Cross-Engine Dry-Run Telemetry (`https://najealp.top`)

On November 11, 2026, executing `pwsh -NoProfile -ExecutionPolicy Bypass -File test\load\run_parallel_suite.ps1 -DryRun` exited cleanly with code `0`, producing zero HTTP failures, zero `401` rejections, and zero Locust exceptions. Table 2 compares the verified Phase 1/2 telemetry alongside an unpaced closed-loop Node reference run (`node_baseline.json`, $[10, 25]$ VUs $\times 5$ s).

Table 2: Observed benchmark metrics across phases and engines targeting `https://najealp.top`.

Execution Stage / Tier	Reqs	Errs / 401	RPS	Mbps	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
Phase 1: JMeter Coverage (`1 VU`)	186	0/0	4.24	—	212.0	322.0	630.0
Phase 1: JMeter Setup Auth	5	0/0	0.39	—	276.0	10727.0	10727.0
Phase 2: JMeter Tier 1 (`10 VUs`, 9.62 s)	141	0/0	14.66	0.85	239.0	916.0	1881.0
Phase 2: JMeter Tier 2 (`25 VUs`, 10.06 s)	358	0/0	35.59	3.53	245.0	884.0	1039.0
Phase 2: JMeter Aggregate Load	499	0/0	25.04	—	244.0	886.0	1065.0
Phase 2: Locust Parallel (`20 VUs`, 23 s)	216	0/0	9.20	—	280.0	1000.0	1300.0
Reference: Node Unpaced (`10 VUs`, 5 s)	473	0/0	93.18	16.74	89.4	170.8	430.0
Reference: Node Unpaced (`25 VUs`, 5 s)	1066	0/0	207.25	35.78	105.4	200.3	382.4

Validation Against Little's Law: Using Tier 1 ($\bar{L}_{\text{resp}} = 331.6$ ms) and Tier 2 ($\bar{L}_{\text{resp}} = 327.62$ ms) from `jmeter_baseline.json` alongside $\mathbb{E}[T_{\text{think}}] = 300$ ms, Eq. (3) predicts steady-state ceilings of $10/0.6316 = 15.83$ RPS and $25/0.6276 = 39.83$ RPS. Measured tier rates (14.66 RPS and 35.59 RPS, inclusive of the 2 s ramp-up window inside each 10 s tier) align closely with theoretical expectations, confirming clean pacing without client socket starvation.

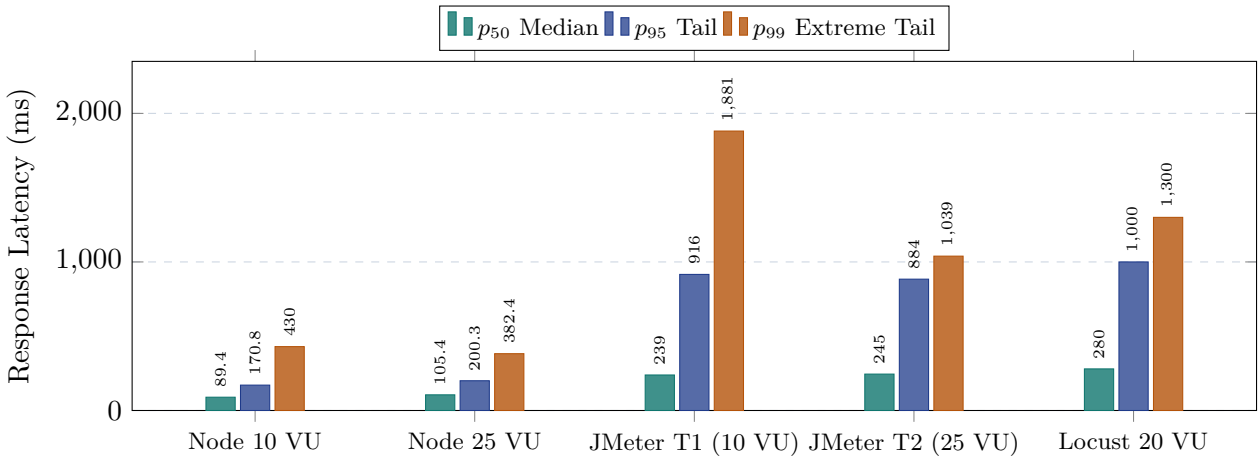


Figure 2: Latency percentile divergence between unpaced read-only Node.js workers and parallel think-time-paced JMeter + write-heavy Locust traffic.

7.2 Route-Level Bottleneck Attribution

Inspecting per-route rankings across `jmeter_baseline.json` and `locust_metrics.json` isolates three physical drivers behind the p_{95}/p_{99} divergence visible in Figure 2:

Table 3: Highest-p_{95} routes observed under parallel Phase 2 load (`JMeter` + `Locust`).

Engine	Route / Transaction Label	Calls	p_{50} (ms)	p_{95} (ms)	Max (ms)
JMeter	`GET /api/users?_where=active&_include`	14	247.0	1935.0	1935.0
JMeter	`GET /api/clefLogs?_page=1&_per_page=10`	8	239.0	1881.0	1881.0
JMeter	`GET /api/events?_sort=eventDate&_order=desc&_page=2&_per_page=5`	11	413.0	1240.0	1240.0
JMeter	`GET /api/events?_sort=eventDate&_order=asc&_page=1&_per_page=10`	12	497.0	1066.0	1066.0
Locust	`GET /api/bulletins [_include=category,author,status]`	14	338.0	1591.3	1591.3
Locust	`DELETE /api/userRsvps/:id [Cleanup]`	10	263.0	1454.5	1454.5
Locust	`GET /api/bulletins [Filtered Page]`	13	473.7	1257.1	1257.1
Locust	`GET [Auth] Verify Regular Token`	11	976.5	1197.3	1197.3
Locust	`POST /api/auth/register [Worst-Case Write]`	17	361.0	1162.0	1162.0

1. **Relational Hydration & Payload Volume:** `/api/events?_sort=eventDate&_order=asc` transmitted 1,026,375 bytes across 12 JMeter calls ($\approx$ 85.5 KB/call, highest total time 7.65 s), and Locust's `/api/bulletins [Filtered Page]` averaged 81,382 bytes per response. Serialization and WAN transfer time push medians near 338–497 ms even without contention.
2. **Event-Loop Blocking via Synchronous Hashing:** Concurrent calls to `POST /api/auth/register` (17 calls, $p_{95} = 1162.0$ ms) execute synchronous `bcrypt.hashSync(password, 10)`, temporarily stalling concurrent event-loop callbacks (explaining why lightweight token checks `GET /api/auth/me` reached $p_{50} = 976.5$ ms during concurrent spawn bursts).
3. **`DELETE`-Journal Serialization:** `DELETE /api/userRsvps/:id` resolves at $p_{50} = 263.0$ ms uncontended, yet jumps 5.5$\times$ to $p_{95} = 1454.5$ ms when queued behind overlapping admin moderation updates and registrations acquiring exclusive SQLite file locks. Meanwhile, network utilization in Tier 2 remained at 3.53 Mbps (5.04% of the 70 Mbps link), proving tail inflation is CPU- and lock-bound rather than bandwidth-saturated.

8 Operational Governance and Reproducibility

8.1 Database Hygiene and Rate-Limiter Controls

After load campaigns execute write bursts (`POST /api/auth/register`, community bulletins, disposable events), synthetic rows must be purged cleanly:

- **Automated API Cleanup:** Both JMeter coverage matrices and Locust's `on_test_stop` hook invoke authenticated `DELETE` endpoints on tracked entities created during the test.
- **Database Restoration (`test/load/teardown.sql`):** Provides an atomic SQLite transaction removing synthetic `loadtest_%@najealp.top`, `test.official%`, and `test.regular%` records alongside dependent `Profile`, `UserStatistic`, `Record`, `UserRsvp`, `Bulletin`, and `Event` rows, followed by `VACUUM` (or VPS snapshot restore via `cp dev.db.bak_pre_stress_* dev.db`).

- **Rate-Limit Awareness:** Operators deploying from benchmark mode back to public production must verify whether `authLimiter` and `contentLimiter` in `api/index.js` should transition from `skip: () => true` back to active per-IP and per-user enforcement.

8.2 Standardized Execution Profiles

Reproducing either the fast verification gate or the full 5-tier stress campaign requires a single PowerShell command from the repository root (ensuring Apache JMeter 5.6.3 and Edge/Chrome are discoverable):

Listing 2: Reproducible invocation profiles for `test/load/run_parallel_suite.ps1`.

```
# 1. Pre-flight verification & DryRun ([10, 25] VUs x 10s, 20 Locust VUs x 24s)
pwsh -NoProfile -ExecutionPolicy Bypass -File test\load\run_parallel_suite.ps1
    -DryRun

# 2. Full 5-tier capacity envelope ([100, 250, 500, 750, 1000] VUs x 30s + 100
    Locust VUs)
pwsh -NoProfile -ExecutionPolicy Bypass -File test\load\run_parallel_suite.ps1
```

Architectural Recommendations Derived from Methodology

1. **Offload Key Derivation:** Replace `bcrypt.hashSync(password, 10)` in registration and user-provisioning paths with asynchronous `await bcrypt.hash(...)` prior to opening any Prisma/SQLite transaction, eliminating Node.js event-loop stalls.
2. **Keep `DELETE`-Mode Boundaries Explicit:** Because project policy intentionally keeps `PRAGMA journal_mode = DELETE`, serialize heavy administrative batch mutations and minimize relational `_include` fan-out on public catalog defaults to keep transaction lock hold times sub-50 ms.
3. **Cache Hot Read Catalogs:** Apply short-lived ETag or in-memory TTL caching to heavy sorted feeds (`/api/events`, `/api/bulletins`) to preserve margin inside the 70 Mbps VPS uplink under 500+ VU tiers.